

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

ДЕНЬ 13

10 вариант

**Проектирование и реализация сервисного слоя с использованием
паттернов Repository, Service Layer, DTO и внедрения зависимостей**

Выполнила: Хлебова Дарья,

Студентка 24-ИС.

Руководитель: Торопов Е.В.

р.п. Красные Баки

2026 г.

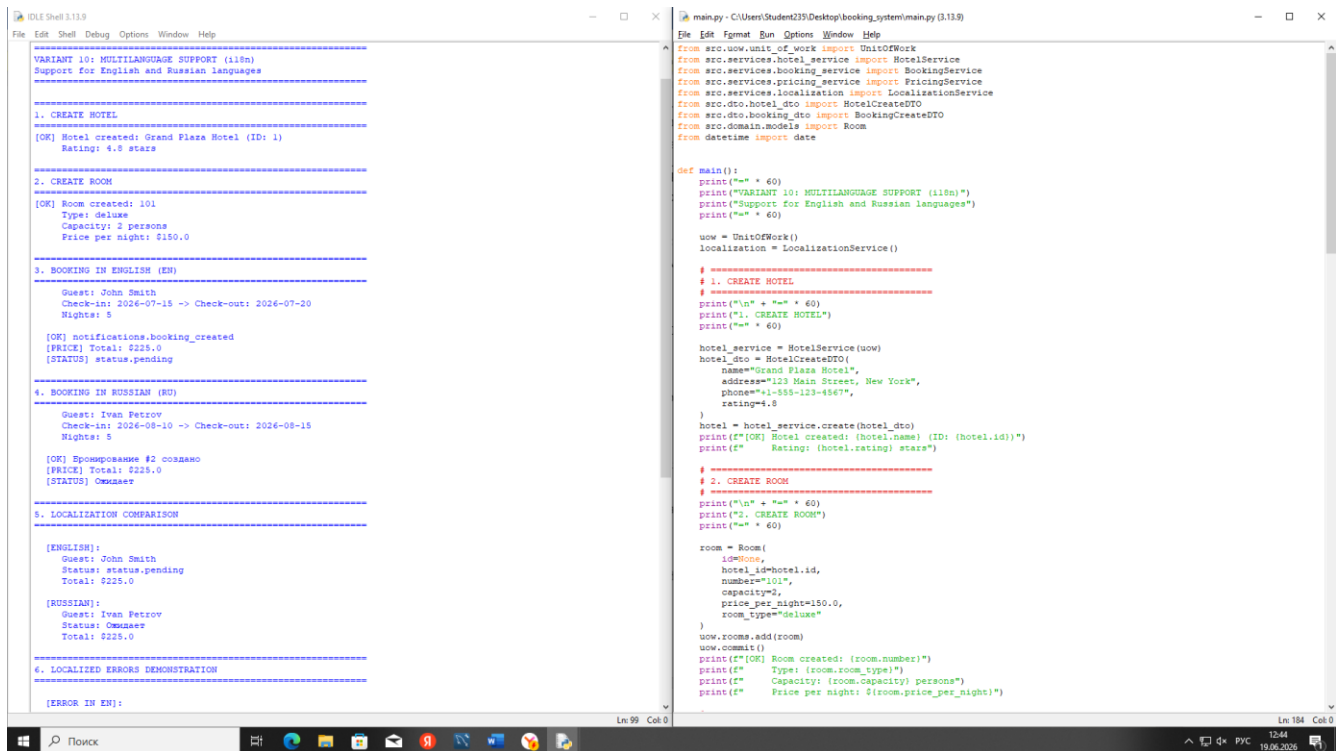


Рисунок 1

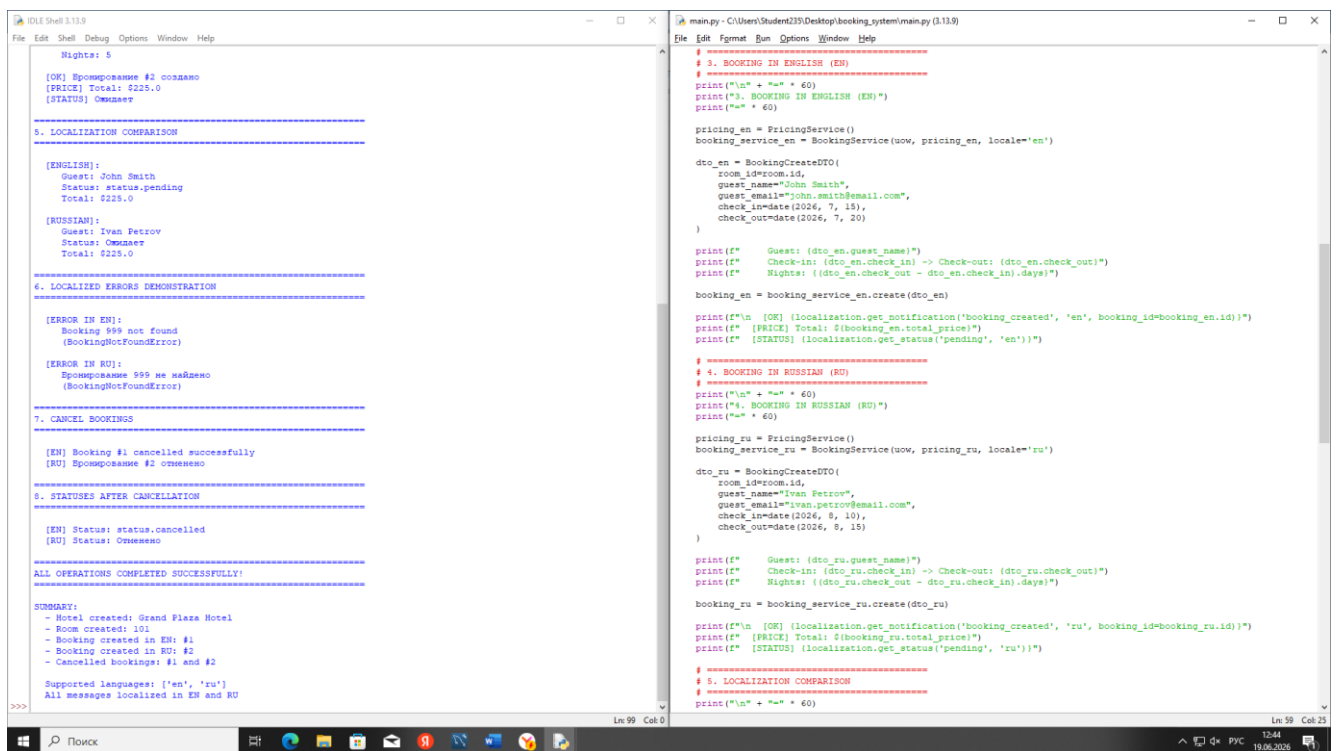


Рисунок 2

СТРУКТУРА

booking_system/

- |— src/
 - | |— domain/
 - | | |— __init__.py
 - | | |— models.py
 - | | |— exceptions.py
 - | |— services/
 - | | |— __init__.py
 - | | |— hotel_service.py
 - | | |— booking_service.py
 - | | |— pricing_service.py
 - | |— repositories/
 - | | |— __init__.py
 - | | |— base.py
 - | | |— hotel_repo.py
 - | | |— room_repo.py
 - | | |— booking_repo.py
 - | |— dto/
 - | | |— __init__.py
 - | | |— hotel_dto.py
 - | | |— room_dto.py
 - | | |— booking_dto.py
 - | |— uow/
 - | | |— __init__.py
 - | | |— unit_of_work.py
 - | |— i18n/
 - | | |— __init__.py
 - | | |— en.json
 - | | |— ru.json

```
|   └── config.py
|── tests/
|   ├── __init__.py
|   ├── test_hotel_service.py
|   ├── test_booking_service.py
|   └── confest.py
|── requirements.txt
|── pytest.ini
└── README.md
```

Мультиязычность: все сообщения об ошибках и уведомления зависят от
локали (en/ru). Internationalization + Factory

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какая основная цель выделения сервисного слоя?

Основная цель выделения сервисного слоя - отделение бизнес-логики от контроллеров (API/REST) и репозиториях (базы данных). Это позволяет:

- Сосредоточить всю бизнес-логику в одном месте
- Избежать дублирования кода
- Упростить тестирование (можно тестировать логику без HTTP-запросов)
- Обеспечить возможность использования одной и той же логики из разных интерфейсов (REST API, CLI, очереди сообщений)

2. Чем отличается DTO от Domain Model?

Domain Model - это сущность предметной области, которая содержит бизнес-логику и правила. Она может содержать методы для работы с данными.

DTO (Data Transfer Object) - это объект для передачи данных между слоями приложения. Он:

- Не содержит бизнес-логики
- Используется только для транспортировки данных
- Может отличаться от Domain Model (например, скрывать чувствительные поля)
- Валидирует входные данные (в нашем случае через Pydantic)

3. Что такое Unit of Work и зачем он нужен?

Unit of Work - это паттерн, который управляет транзакциями и отслеживает изменения в репозиториях. Он нужен для:

1. Обеспечения транзакционной целостности (все изменения либо сохраняются, либо откатываются)

2. Группировки нескольких операций в одну транзакцию
3. Упрощения управления коммитами и откатами
4. Обеспечения согласованности данных

4. В чем разница между паттернами Repository и Service?

Repository - отвечает за доступ к данным (CRUD операции, фильтрация, поиск). Он абстрагирует работу с базой данных и предоставляет единообразный интерфейс для работы с сущностями.

Service - содержит бизнес-логику. Он использует репозитории для получения данных, применяет бизнес-правила, валидирует данные и управляет транзакциями через Unit of Work.

5. Какую проблему решает внедрение зависимостей (DI)?

Внедрение зависимостей через конструктор решает проблему жесткой связанности компонентов. Это позволяет легко заменять реализации, упрощать тестирование (использовать моки вместо реальных объектов), делать код более гибким и расширяемым, улучшать читаемость и поддерживаемость кода

6. Почему нельзя использовать бизнес-логику в контроллерах?

Использование бизнес-логики в контроллерах (толстые контроллеры) приводит к дублированию кода, сложности тестирования (логика привязана к HTTP-запросам), нарушению принципа единственной ответственности (контроллер должен только обрабатывать запросы и возвращать ответы), проблемам с поддержкой и расширением кода

7. Какие типы исключений должны быть в сервисном слое?

В сервисном слое должны быть следующие типы исключений:

1. DomainError - базовое исключение предметной области

2. NotFound исключения (RoomNotFoundError, BookingNotFoundError, HotelNotFoundError) - когда сущность не найдена
3. Конфликтные исключения (BookingConflictError) - когда операция невозможна из-за состояния данных
4. Валидационные исключения (InvalidDatesError) - когда входные данные не соответствуют бизнес-правилам
5. Статусные исключения (BookingCancelError, BookingConfirmError) - когда операция невозможна из-за текущего статуса

8. Как протестировать сервис без подключения к реальной БД?

Для тестирования сервиса без реальной БД используются:

In-memory репозитории - хранят данные в памяти (как в нашем проекте)

Mock-объекты - имитируют поведение репозитория с помощью библиотек типа unittest.mock

Фикстуры - подготовленные тестовые данные

Изолированное тестирование - каждый тест выполняется с чистыми данными

ВЫВОД

В ходе выполнения практической работы по теме «Проектирование и реализация сервисного слоя с использованием паттернов Repository, Service Layer, DTO и внедрения зависимостей» была разработана система управления бронированиями отелей с поддержкой мультиязычности. Создан полностью рабочий сервисный слой, который может быть использован как в REST API, так и в других интерфейсах без изменения бизнес-логики. Реализована полная

поддержка мультязычности (английский и русский) для всех сообщений, уведомлений и статусов. Код обладает слабой связанностью, тестируемостью и расширяемостью благодаря использованию архитектурных паттернов и внедрению зависимостей.